FASE 2: DEV SÊNIOR

FLASHCARDS DAS MÉTRICAS OO (SUÍTE CK)

ASSUNTO 1: WMC (Weighted Methods per Class) — Complexidade da Classe

Flashcard 1.1 - Definição e Propósito: Significa Weighted Methods per Class (Métodos Ponderados por Classe). É uma métrica da suíte CK usada para mensurar a complexidade interna de uma classe específica através da medição de seus métodos.

Flashcard 1.2 - Por que é Importante / Vantagens: Avalia o esforço necessário para compreender, alterar, testar e manter uma classe. Um WMC baixo indica uma classe simples e coesa. Um WMC muito alto é um alerta de que a classe acumulou responsabilidades demais, violando o Princípio de Responsabilidade Única (SRP).

Flashcard 1.3 - Como Calcula / Fórmulas: Na sua fórmula tradicional mais adotada (simplificada), assume-se peso 1 para todos os métodos. Assim, o cálculo consiste na contagem simples da quantidade total de métodos declarados diretamente na classe:

```
WMC= quantidade de métodos
```

Flashcard 1.4 - Exemplo Prático de Código:

```
// WMC = 4 (Classe simples e saudável)
class ContaBancaria {
    void sacar() { ... }
    void depositar() { ... }
    void transferir() { ... }
    void consultarSaldo() { ... }
}
```

ASSUNTO 2: DIT (Depth of Inheritance Tree) — Profundidade de Herança

Flashcard 2.1 - Definição e Propósito: Significa Depth of Inheritance Tree (Profundidade da Árvore de Herança). É a métrica utilizada para mensurar o tamanho da cadeia de herança de uma classe dentro da hierarquia do sistema.

Flashcard 2.2 - Por que é Importante / Vantagens: Mede o nível de reutilização de código e a dispersão lógica do sistema. Um valor de DIT moderado é desejável, pois indica bom reuso. Já um DIT muito alto fragmenta as funções e espalha o comportamento em muitas classes pai, tornando o entendimento do fluxo muito difícil.

Flashcard 2.3 - Como Calcula / Fórmulas: O cálculo consiste em achar a distância máxima da classe analisada até a classe ancestral que serve de raiz (topo da árvore). A classe raiz começa no nível 0:

```
DIT (CLASSE RAIZ) =0
DIT(SUBCLASSE DIRETA) = DIT(CLASSE PAI) + 1
```

Flashcard 2.4 - Exemplo Prático de Código:

```
class Pessoa { } // DIT = 0 (Raiz)

class Funcionario extends Pessoa { } // DIT = 1

class Professor extends Funcionario { } // DIT = 2 (Dois passos até a raiz)
```

ASSUNTO 3: NOC (Number of Children) — Quantidade de Subclasses

Flashcard 3.1 - Definição e Propósito: Significa Number of Children (Número de Filhos). É a métrica que quantifica quantas subclasses estendem diretamente uma classe específica.

Flashcard 3.2 - Por que é Importante / Vantagens: Um NOC alto indica que a classe pai é muito importante e genérica no projeto, servindo como uma excelente base para reutilização de código. Contudo, NOC alto exige testes rígidos, pois qualquer alteração na classe pai afetará de forma imediata o comportamento de todos os seus filhos.

Flashcard 3.3 - Como Calcula / Fórmulas: Para calcular, faz-se a contagem exclusiva de quantas subclasses declaram herança direta (usando extends) da classe em questão:

```
NOC = QUANTIDADE DE FILHOS DIRETOS
```

Flashcard 3.4 - Exemplo Prático de Código:

```
class Veiculo { } // NOC = 3 (Possui três subclasses herdando dele)

class Carro extends Veiculo { }
class Moto extends Veiculo { }
class Caminhao extends Veiculo { }
```

ASSUNTO 4: CBO (Coupling Between Objects) — Acoplamento de Classes

Flashcard 4.1 - Definição e Propósito: Significa Coupling Between Objects (Acoplamento Entre Objetos). É a métrica que calcula a dependência estrutural de uma classe em relação a outras classes do sistema.

Flashcard 4.2 - Por que é Importante / Vantagens: O CBO baixo é altamente desejável. Ele garante a independência das classes, fazendo com que uma alteração em um ponto do código não propague erros em cascata pelo resto do sistema. Além disso, classes com baixo CBO são muito mais fáceis de testar de forma unitária e de serem reutilizadas.

Flashcard 4.3 - Como Calcula / Fórmulas: A contagem rastreia o número de outras classes das quais a classe atual depende de forma explícita. Conta-se as classes externas usadas como atributos, tipos de parâmetros de métodos, herança ou instanciadas diretamente no escopo:

```
cbo= quantidade de classes externas acopladas
```

Flashcard 4.4 - Exemplo Prático de Código:

```
// CBO = 3 (A classe Pedido depende de Cliente, Produto e Pagamento para funcionar)
class Pedido {
    Cliente cliente;
    Produto produto;
    Pagamento pagamento;
}
```

ASSUNTO 5: RFC (Response For a Class) — Complexidade de Resposta

Flashcard 5.1 - Definição e Propósito: Significa Response For a Class (Conjunto de Resposta de uma Classe). Ela mede a complexidade de resposta e o acoplamento dinâmico da classe, mapeando todos os métodos que podem ser disparados a partir de uma única chamada externa.

Flashcard 5.2 - Por que é Importante / Vantagens: Avalia o quão complexo será testar e depurar a classe. Um RFC alto significa que a classe possui uma dependência de execução muito espalhada, dificultando simulações de teste (exigindo mockar muitos comportamentos) e tornando o comportamento difícil de prever.

Flashcard 5.3 - Como Calcula / Fórmulas: Soma-se o número de métodos internos declarados na própria classe com todos os métodos de outras classes externas que são chamados de dentro das funções locais:

```
rfc= m+r
```

Onde m representa a quantidade de métodos da própria classe, e r a quantidade de métodos externos invocados.

Flashcard 5.4 - Exemplo Prático de Código:

```
// RFC = 4 (1 método local + 3 métodos externos chamados)
class Pedido {
    void realizarCompra() {
        validar();          // Método externo
        calcularTotal();    // Método externo
        gerarNota();        // Método externo
    }
```

ASSUNTO 6: LCOM (Lack of Cohesion in Methods) — Falta de Coesão nos Métodos

Flashcard 6.1 - Definição e Propósito: Significa Lack of Cohesion in Methods (Falta de Coesão nos Métodos). É a métrica que avalia a coesão estrutural de uma classe, calculando quantos métodos da classe NÃO compartilham o uso de atributos comuns da própria classe.

Flashcard 6.2 - Por que é Importante / Vantagens: Nós queremos um LCOM baixo (o que significa alta coesão). Classes com alta coesão são mais fáceis de testar e focar. Um LCOM alto indica que a classe é um agrupado de métodos sem nenhuma ligação de dados, devendo ser dividida em novas classes de responsabilidades únicas.

Flashcard 6.3 - Como Calcula / Fórmulas: Comparam-se todos os pares de métodos da classe. Se p for o número de pares que não compartilham atributos, e q o número de pares que compartilham pelo menos um atributo:

```
se p> q = lcom= p-q
se p<= q = lcom =0
```

Flashcard 6.4 - Exemplo Prático de Código:

```
// LCOM = 0 (Coesa/Desejável) - Todos compartilham o atributo 'saldo'
class Conta {
    double saldo;
    void sacar() { ... }
    void depositar() { ... }
}

// LCOM Alto (Falta de Coesão/Ruim) - Métodos não dividem nenhum atributo lógico
class SistemaGeral {
    void emitirRelatorio() { ... }
    void atualizarEstoque() { ... }
}
```